Lab 3

Four by Sixteen Multiplexer

By: Ben Robles & Robert Stevenson
Group U

ECE-3300L
Summer 2026

Date: 07/01/2026

Design:

XDC Snippet:

```
# Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }];
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}];


##Switches

set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }];
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }];
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }];
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { SW[3] }];
..
set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { SW[12] }];
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }];
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }];
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }];


## LEDs

set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { LED0 }]; #IO_L18P_T2_A24_15 Sch=led[0]


##Buttons

set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { rst }]; #IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { btnU }]; #IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { btnL }]; #IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { btnR }]; #IO_L10N_T1_D15_14 Sch=btnr
set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { btnD }]; #IO_L9N_T1_DQS_D13_14 Sch=btnd
```

Simulation:

Two by One Multiplexer Test Bench

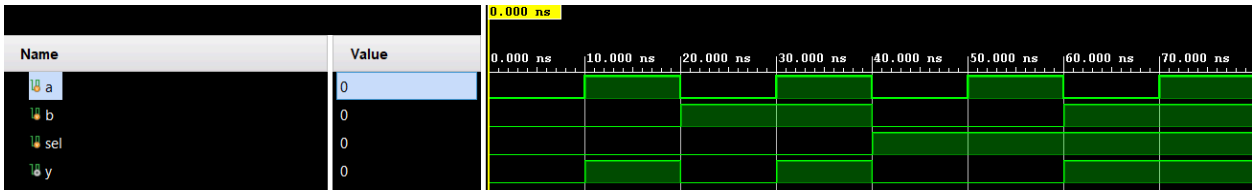
```
module mux2x1_tb(

                        );

    reg a, b, sel;
    wire y;
    reg [1:0] inval;
    integer i, j;
```

```
    mux2x1 mx_tb(.a(a), .b(b), .sel(sel), .y(y));

    initial begin
        inval = 2'b00;
        a = 0;
        b = 0;
        sel = 0;
        for(i=0; i<2; i=i+1)begin
            sel = i;
            for(j=0; j<4; j=j+1)begin
                inval = j;
                a = inval[0];
                b = inval[1];
                #10;
            end
        end
        $finish;
    end
endmodule
```

Description:

For the two by one multiplexer I ran nested for loops which would send all four possible combinations of inputs to the mux2x1 module for both sel values.

Waveform:



Sixteen by Four Multiplexer Test Bench:

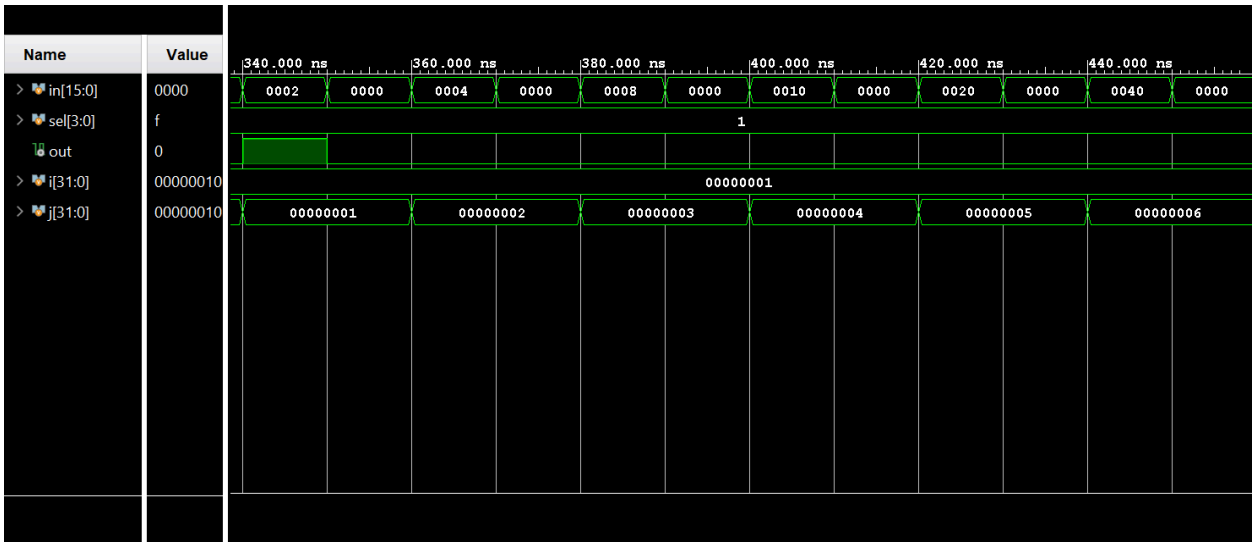
```
module mux16x1_tb(

                );

    reg [15:0] in;
    reg [3:0] sel;
    wire out;

    integer i, j;

      mux16x1 mx16_tb(.in(in), .sel(sel), .out(out));

    initial begin
        in <= 16'd0;
        sel <= 4'd0;
```

```
        for(i=0; i<16; i=i+1)begin
            sel <= i;
            for(j=0; j<16; j=j+1)begin
                in[j] = 1;
                #10;
                in[j] = 0;
                #10;
            end
        end
        $finish;
    end
endmodule
```

Description:

For the sixteen by one multiplexer I ran nested for loops that would turn on all sixteen inputs into the multiplexer for all sixteen possible selection values.

Waveform:



Debounce Test Bench:

```
module top_mux_tb(

                );

    reg clk_tb;
    reg rst_tb;
    reg [15:0] sw_tb;

    reg btnU_tb;
    reg btnD_tb;
    reg btnL_tb;
```

```
reg btnR_tb;

wire led0_tb;

integer i;

top_mux_lab3 top_tb(
                    .clk(clk_tb),
                    .rst(rst_tb),
                    .SW(sw_tb),
                    .btnU(btnU_tb),
                    .btnD(btnD_tb),
                    .btnL(btnL_tb),
                    .btnR(btnR_tb),
                    .LED0(led0_tb)
                );

initial
   begin
      clk_tb = 1'b0;
   end

always
   begin
      #5 clk_tb = ~clk_tb;
   end

initial
      begin
         rst_tb = 1'b1;
         sw_tb = 16'b0000_0000_0000_0000;
         btnU_tb = 1'b0;
         btnD_tb = 1'b0;
         btnL_tb = 1'b0;
         btnR_tb = 1'b0;

         @(negedge clk_tb);
         rst_tb = 1'b0;
         sw_tb = 16'b0000_0000_0000_0001;

         @(negedge clk_tb);
         if(led0_tb !== 1'b1)
            begin
               $display("FAILED DEFAULT TEST");
               $fatal;
            end
         else
            begin
               $display("PASSED DEFAULT TEST");
            end

         @(negedge clk_tb);
         sw_tb = 16'b0000_0000_0000_0010;
         btnD_tb = 1'b1;
```

```
for(i = 0; i < 6; i = i + 1)
   begin
      @(posedge clk_tb);
   end

if(led0_tb !== 1'b1)
   begin
      $display("FAILED BTN D TEST");
      $fatal;
   end
else
   begin
      $display("PASSED BTN D TEST");
   end

@(negedge clk_tb);
sw_tb = 16'b0000_0000_0000_1000;
btnR_tb = 1'b1;

for(i = 0; i < 6; i = i + 1)
   begin
      @(posedge clk_tb);
   end

if(led0_tb !== 1'b1)
   begin
      $display("FAILED BTN D + BTN R TEST");
      $fatal;
   end
else
   begin
      $display("PASSED BTN D + BTN R TEST");
   end

@(negedge clk_tb);
sw_tb = 16'b0000_0000_1000_0000;
btnL_tb = 1'b1;

for(i = 0; i < 6; i = i + 1)
   begin
      @(posedge clk_tb);
   end

if(led0_tb !== 1'b1)
   begin
      $display("FAILED BTN D + BTN R + BTN L TEST");
      $fatal;
   end
else
   begin
      $display("PASSED BTN D + BTN R + BTN L TEST");
   end
```

```
            @(negedge clk_tb);
            sw_tb = 16'b1000_0000_0000_0000;
            btnU_tb = 1'b1;

            for(i = 0; i < 6; i = i + 1)
               begin
                  @(posedge clk_tb);
               end

            if(led0_tb !== 1'b1)
               begin
                  $display("FAILED BTN D + BTN R + BTN L + BTN U TEST");
                  $fatal;
               end
            else
               begin
                  $display("PASSED BTN D + BTN R + BTN L + BTN U TEST");
               end

            @(negedge clk_tb);
            rst_tb = 1'b1;

            for(i = 0; i < 3; i = i + 1)
               begin
                  @(posedge clk_tb);
               end

            if(led0_tb !== 1'b0)
               begin
                  $display("FAILED RESET TEST");
                  $fatal;
               end
            else
               begin
                  $display("PASSED RESET TEST");
               end

            $display("ALL TESTS PASSED");
            $finish;
         end
endmodule
```

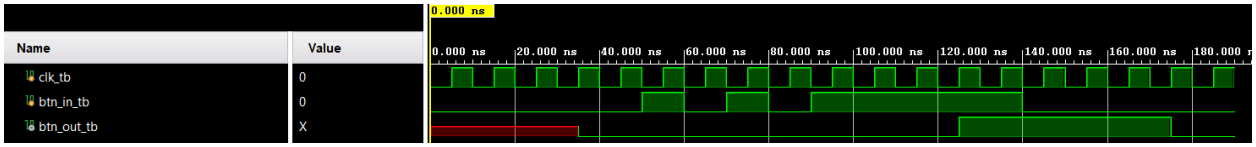
Description:

To verify the debounce implementation, an initial check verifies the output goes low after three low input cycles. Then, a debounce test toggles the input to verify the output does not change. After that, the input is held high for three cycles and tests that the output goes high. Finally, the input goes back low and the output is verified. Each test is self checking with a console output and failure breaks.

Output:

```
# run 1000ns
PASSED INITIAL STATE TEST
PASSED DEBOUNCE TEST
PASSED OUTPUT HIGH TEST
PASSED OUTPUT LOW TEST
$finish called at time : 190 ns : File "D:/School/ECE3300/ECE3300_Lab3_GroupU
```

Waveform:



Toggle Switch Test Bench:

```
module toggle_switch_tb(

                    );

    reg clk_tb;
    reg rst_tb;
    reg btn_raw_tb;
    wire state_tb;

    integer i;

    toggle_switch ts_tb(
                        .clk(clk_tb),
                        .rst(rst_tb),
                        .btn_raw(btn_raw_tb),
                        .state(state_tb)
                    );

    initial
        begin
            clk_tb = 1'b0;
        end

    always
        begin
            #5 clk_tb = ~clk_tb;
        end

    initial
            begin
                rst_tb = 1'b1;
                btn_raw_tb = 1'b0;

                for(i = 0; i < 3; i = i + 1)
```

```
            begin
                @(posedge clk_tb);
            end

        @(negedge clk_tb);
        rst_tb = 1'b0;
        btn_raw_tb = 1'b1;


        for(i = 0; i < 6; i = i + 1)
            begin
                @(posedge clk_tb);
            end

            @(negedge clk_tb);

        if(state_tb !== 1'b1)
            begin
                $display("FAILED TO TOGGLE");
                $fatal;
            end
        else
            begin
                $display("PASSED TEST TOGGLE");
            end

        @(negedge clk_tb);
        rst_tb = 1'b1;
        btn_raw_tb = 1'b0;

        @(negedge clk_tb);
        if(state_tb !== 1'b0)
            begin
                $display("FAILED TO RESET");
                $fatal;
            end
        else
            begin
                $display("PASSED TEST RESET");
            end
          $finish;
      end
endmodule
```

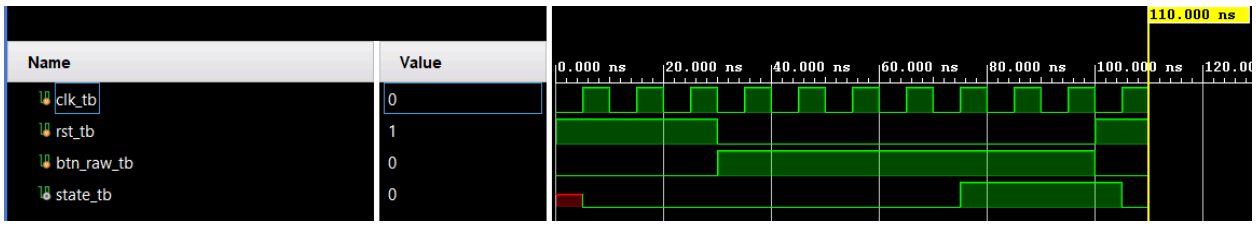
Description:

To verify the toggle switch implementation i reset the state initially, let the debounce output settle, and activated the toggle. Within a few cycles, I verified that the state changed. After this, I activated the reset button again, verifying that the state goes back to low. Each section is self checking with a console output and failure breaks.

Output:

```
# run 1000ns
PASSED TEST TOGGLE
PASSED TEST RESET
$finish called at time : 110 ns : File "D:/School/ECE3300/ECE3300_Lab3_GroupU
```

Waveform:



Top Module Test Bench:

```
module top_mux_tb(

        );

    reg clk_tb;
    reg rst_tb;
    reg [15:0] sw_tb;

    reg btnU_tb;
    reg btnD_tb;
    reg btnL_tb;
    reg btnR_tb;

    wire led0_tb;

    integer i;

    top_mux_lab3 top_tb(
                    .clk(clk_tb),
                    .rst(rst_tb),
                    .SW(sw_tb),
                    .btnU(btnU_tb),
                    .btnD(btnD_tb),
                    .btnL(btnL_tb),
                    .btnR(btnR_tb),
                    .LED0(led0_tb)
                );

    initial
        begin
            clk_tb = 1'b0;
        end

    always
```

```
    begin
        #5 clk_tb = ~clk_tb;
    end

initial
        begin
            rst_tb = 1'b1;
            sw_tb = 16'b0000_0000_0000_0000;
            btnU_tb = 1'b0;
            btnD_tb = 1'b0;
            btnL_tb = 1'b0;
            btnR_tb = 1'b0;

            @(negedge clk_tb);
            rst_tb = 1'b0;
            sw_tb = 16'b0000_0000_0000_0001;

            @(negedge clk_tb);
            if(led0_tb !== 1'b1)
                begin
                    $display("FAILED DEFAULT TEST");
                    $fatal;
                end
            else
                begin
                    $display("PASSED DEFAULT TEST");
                end

            @(negedge clk_tb);
            sw_tb = 16'b0000_0000_0000_0010;
            btnD_tb = 1'b1;

            for(i = 0; i < 6; i = i + 1)
                begin
                    @(posedge clk_tb);
                end

            if(led0_tb !== 1'b1)
                begin
                    $display("FAILED BTN D TEST");
                    $fatal;
                end
            else
                begin
                    $display("PASSED BTN D TEST");
                end

            @(negedge clk_tb);
            sw_tb = 16'b0000_0000_0000_1000;
            btnR_tb = 1'b1;

            for(i = 0; i < 6; i = i + 1)
                begin
                    @(posedge clk_tb);
```

```
    end

if(led0_tb !== 1'b1)
    begin
        $display("FAILED BTN D + BTN R TEST");
        $fatal;
    end
else
    begin
        $display("PASSED BTN D + BTN R TEST");
    end

@(negedge clk_tb);
sw_tb = 16'b0000_0000_1000_0000;
btnL_tb = 1'b1;

for(i = 0; i < 6; i = i + 1)
    begin
        @(posedge clk_tb);
    end

if(led0_tb !== 1'b1)
    begin
        $display("FAILED BTN D + BTN R + BTN L TEST");
        $fatal;
    end
else
    begin
        $display("PASSED BTN D + BTN R + BTN L TEST");
    end

@(negedge clk_tb);
sw_tb = 16'b1000_0000_0000_0000;
btnU_tb = 1'b1;

for(i = 0; i < 6; i = i + 1)
    begin
        @(posedge clk_tb);
    end

if(led0_tb !== 1'b1)
    begin
        $display("FAILED BTN D + BTN R + BTN L + BTN U TEST");
        $fatal;
    end
else
    begin
        $display("PASSED BTN D + BTN R + BTN L + BTN U TEST");
    end

@(negedge clk_tb);
rst_tb = 1'b1;

for(i = 0; i < 3; i = i + 1)
```

```
            begin
                @(posedge clk_tb);
            end

        if(led0_tb !== 1'b0)
            begin
                $display("FAILED RESET TEST");
                $fatal;
            end
        else
            begin
                $display("PASSED RESET TEST");
            end

        $display("ALL TESTS PASSED");
        $finish;
    end
endmodule
```

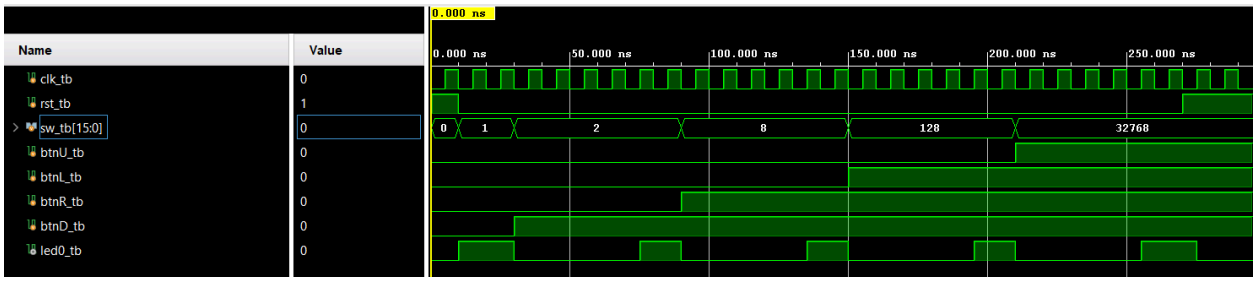
Description:

Since the internal modules are verified indepentently I focused on testing the button toggles over each individual output. I compounded the button activations to move through select 0000 –> 0001 → 0011 → 0111 → 1111. This meant verifying the activation of sw[0,1,3,7,15] in that order. Finally, I activated the reset button to verify all toggles were disabled and the output went low.

Output:

```
# run 1000ns
PASSED DEFAULT TEST
PASSED BTN D TEST
PASSED BTN D + BTN R TEST
PASSED BTN D + BTN R + BTN L TEST
PASSED BTN D + BTN R + BTN L + BTN U TEST
PASSED RESET TEST
ALL TESTS PASSED
$finish called at time : 295 ns : File "D:/School/ECE3300/ECE3300_Lab3_GroupU
```

Waveform:



Implementation:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	Bonded IOB (210)	BUFGCTRL (32)
N top_mux_lab3	15	31	2	1	9	15	23	1

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	7.424 ns	Worst Hold Slack (WHS):	0.162 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	30	Total Number of Endpoints:	30	Total Number of Endpoints:	32

All user specified timing constraints are met.

Group Video:

https://youtu.be/v0cNMURfHe0

Contributions:

Ben Robles: Top, toggle, and debounce simulation
Robert Stevenson: Design implementation/XDC file, Mux simulation

Reflections:

Robert Stevenson: In this lab I had the opportunity to work with the system clock on the Artix-7 chip for the first time. This was initially challenging getting the constraints file to work properly with the design files, but eventually it all came together.

Ben Robles: I enjoyed diving into the IP level for this lab, having been given the design and attempting to create as thorough a test bench as possible. This definetely took a long time and a lot of thinking to cover all the edge cases, but it was extremely insightful to work through this process.